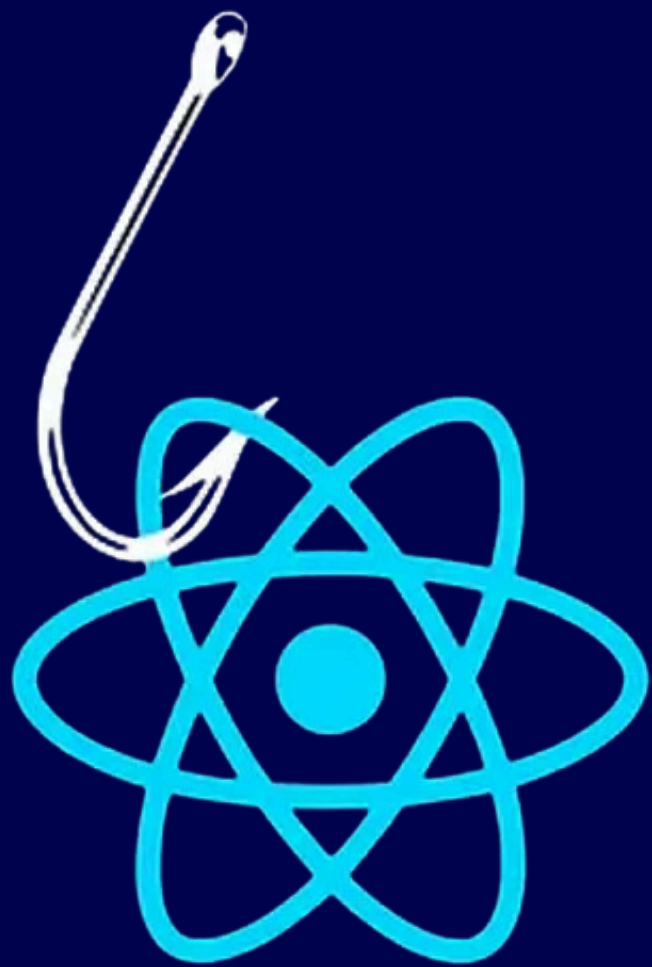




React

New Hooks

19



Introduction

React 19 introduces new hooks that simplify common challenges in React development, especially around state management, form handling, and asynchronous updates.

Here are the new additions that comes with this latest update:

- **useFormState ()**
- **useFormStatus ()**
- **useActionState ()**
- **useOptimistic ()**
- **use ()**

We will be learning how the procecesses for these hooks were handled before and how these hooks make all the differences.

useFormState ()

Before React 19, managing form data usually required setting up a **useState** hook for each form field or a single state object with multiple fields. Custom onChange are usually set up with manual update of the state for each input field, which added complexity to the code.

With the **useFormstate()** hook in React 19, form states can be managed more cleanly, as it keeps track of all form inputs and updates them automatically when fields change.

It simplifies the tracking and updating of form field values by consolidating form data into a **single state object**.

This hook allows for real-time tracking of input values without needing separate **useState** handlers for each field. It also helps keep the form state in sync, especially when dealing with complex forms.

For example: older way of handling form field states

```
1 import { useState } from 'react';
2
3 function ProfileForm() {
4   const [name, setName] = useState('');
5   const [email, setEmail] = useState('');
6
7   const handleSubmit = async (e) => {
8     e.preventDefault();
9     await fetch('/api/profile', { method: 'POST',
10       body: JSON.stringify({ name, email }) });
11   };
12
13   return (
14     <form onSubmit={handleSubmit}>
15       <input
16         name="name"
17         value={name}
18         onChange={(e) => setName(e.target.value)}
19         placeholder="Name"
20       />
21       <input
22         name="email"
23         value={email}
24         onChange={(e) => setEmail(e.target.value)}
25         placeholder="Email"
26       />
27       <button type="submit">Submit</button>
28     </form>
29   );
30 }
```

The form's name and email fields are tracked separately, requiring unique useState calls.

For example: with useFormState() hook



```
1 import { useFormState } from 'react';
2
3 function ProfileForm() {
4   const [formData, setFormData] = useFormState({
5     name: '',
6     email: '',
7   });
8
9   const handleChange = (e) => {
10     setFormData({ ...formData, [e.target.name]: e.target.value
11   });
12
13   const handleSubmit = () => {
14     // Handle form submission with the entire formData object
15   };
16
17   return (
18     <form onSubmit={handleSubmit}>
19       <input name="name" value={formData.name} onChange=
20         {handleChange} />
21       <input name="email" value={formData.email} onChange=
22         {handleChange} />
23       <button type="submit">Submit</button>
24     </form>
25   );
26 }
```

All form fields managed in one state object.

useFormStatus ()

Before React 19, form states are usually managed by setting up multiple state variables (e.g., loading, error, success, submitting, submitted, idle) to handle form status and submission states.

This often led to complex code to track and render the form state, such as disabling the submit button during submission or showing loading indicators.

In React 19, the useFormStatus() hook provides a more streamlined way to manage these statuses. It lets easy tracking of whether a form is submitting, submitted, idle, or has encountered an error.

This makes managing the visual feedback of form interactions (e.g., disabling buttons or showing loading spinners) more straightforward.

For example: older way of handling form submission

```
1 const UserProfile = () => {
2   const [name, setName] = useState("");
3   const [pending, setPending] = useState(false);
4
5   const handleSubmit = async (event) => {
6     event.preventDefault();
7     setPending(true);
8     await fetch('/api/profile', { method: 'POST',
9       body: JSON.stringify(formData) });
10    setPending(false);
11    setName("");
12  };
13
14  return (
15    <form onSubmit={handleSubmit}>
16      <input
17        type="text"
18        value={name}
19        placeholder="Enter your name"
20        onChange={(e) => setName(e.target.value)}
21      />
22      <button type="submit">Submit</button>
23      {pending} && <p>Submitting {name} ... </p>
24    </form>
25  );
26 };
```

Form submission status
being handled using the
useState hook.

Status update in the UI

Example: with `useFormStatus()` hook

When you use **`useFormStatus`**, you receive an object with properties like **`pending`**, **`data`**, **`method`**, and **`action`**. Each of these properties helps manage and respond to the form's state, improving the UX by allowing you to show loading indicators, success messages, or error handling.

- **`pending`**: A boolean that is true when the form is in the process of submitting and false otherwise.
- **`data`**: Contains any response data from the form submission.
- **`method`**: Indicates the HTTP method used for the form (like POST or GET).
- **`action`**: The URL or endpoint to which the form is being submitted.

In this example, we will use **`useFormStatus`** to disable the submit button while the form is being submitted, show a loading indicator during submission, and display a success or error message once the submission is complete.



```
1 import { useFormStatus } from 'react';
2
3 function LoginForm() {
4   const { pending, data, method, action } = useFormStatus();
5
6   const handleSubmit = async (event) => {
7     event.preventDefault();
8     // Simulate form submission with fetch or other method
9   };
10
11  return (
12    <form method="POST" action="/login" onSubmit={handleSubmit}>
13      <h3>Login</h3>
14      <label>
15        Username:
16        <input name="username" type="text" required />
17      </label>
18      <label>
19        Password:
20        <input name="password" type="password" required />
21      </label>
22      <button type="submit" disabled={pending}>
23        {pending ? 'Logging in...' : 'Login'}
24      </button>
25
26      { /* Display success message if data is available */ }
27      {data && <p>Welcome back, {data.username}!</p>}
28
29      { /* Display form metadata for debugging purposes */ }
30      <p>Form Method: {method}</p>
31      <p>Form Action: {action}</p>
32    </form>
33  );
34 }
```

Form submission status handled using the useFormStatus hook

In this example:

- The **pending** property is used to **disable** the submit button while the form is in the middle of submission. When **pending** is **true**, it shows a loading message, providing feedback to the user that the form is being processed. Once the form has
- submitted successfully, **data** will hold any response information returned from the submission. Here, if **data** is populated (e.g., it has a username field), we display a welcome message to the user. **method** and **action** provide
- useful **metadata** about the form. In this example, they are shown as debugging information to indicate the **HTTP** method and action **URL** being used.

useState ()

In earlier versions of React, handling **asynchronous action states** often required using multiple **useState** variables to track the **status** of each operation:

```
1 import { useState } from 'react';
2
3 function SubmitButton() {
4   const [isSubmitting, setIsSubmitting] = useState(false);
5   const [error, setError] = useState(null);
6   const [success, setSuccess] = useState(false);
7
8   const handleSubmit = async () => {
9     setIsSubmitting(true);
10    setError(null);
11    setSuccess(false);
12    try {
13      await fetch('/api/submit', { method: 'POST' });
14      setSuccess(true);
15    } catch (err) {
16      setError('Submission failed');
17    } finally {
18      setIsSubmitting(false);
19    }
20  };
21
22  return (
23    <div>
24      <button onClick={handleSubmit} disabled={isSubmitting}>
25        {isSubmitting ? 'Submitting...' : 'Submit'}
26      </button>
27      {error && <p>{error}</p>}
28      {success && <p>Success!</p>}
29    </div>
30  );
31 }
```

You need separate state variables (**isSubmitting**, **error**, **success**) to represent each possible state of the action.

The code could become verbose and harder to manage, especially as more action states are added or if multiple actions are performed in the component.

The **useActionState** is a generalized hook designed to manage the lifecycle of these **actions** or **tasks** in a more streamlined way, by providing a standardized way to manage and respond to each stage of the operations.

It helps maintain a clear, controlled flow in UI updates, especially when actions need to track multiple statuses like "**idle**," "**loading**," "**success**," or "**error**."

Using our example above, the **useActionState** hook will help simplify this pattern by encapsulating the various states of an action into a **single hook**, allowing you to handle the loading, success, and error states more concisely.

Updating the former example to use **useActionState** hook:

```
1 import { useActionState } from 'react';
2
3 function SubmitButton() {
4   const [submitStatus, performSubmit] = useActionState(async () => {
5     await fetch('/api/submit', { method: 'POST' });
6   });
7
8   return (
9     <div>
10       <button onClick={performSubmit} disabled={submitStatus.isLoading}>
11         {submitStatus.isLoading ? 'Submitting...' : 'Submit'}
12       </button>
13       {submitStatus.error && <p>{submitStatus.error.message}</p>}
14       {submitStatus.isSuccess && <p>Success!</p>}
15     </div>
16   );
17 }
```

Form submission statuses encapsulated into a single hook

With `useActionState`:

- **`submitStatus`** is an object that contains properties like **`isLoading`**, **`isSuccess`**, and **`error`**, which you can use to conditionally render UI based on the action's current state.
- **`performSubmit`** is the function to invoke the action, which automatically updates `submitStatus`. There's no need for additional state variables or manual error handling in the UI logic, reducing code complexity and making it more readable.

useOptimistic ()

Before React 19, to create optimistic UI updates, where changes appear instantly **before** server confirmation, we often used a mix of state variables and asynchronous functions.

For instance, we'd set a state variable to indicate an immediate UI update, then send the API request and update the state again based on the response.

This approach was functional but required a lot of boilerplate code and error handling to ensure consistency between UI and server state.

in React 19, the **useOptimistic()** hook simplifies this by providing an easy way to handle optimistic updates. This hook allows you to update the UI **immediately**, even as the actual server confirmation is **pending**.

If the server request fails, **useOptimistic()** can be configured to **revert** the state, thus providing a smooth experience with minimal code.

For example: older way of handling optimistic update

```
1 import { useState } from 'react';
2
3 function LikeButton({ postId }) {
4   const [isLiked, setIsLiked] = useState(false);
5   const [error, setError] = useState(null);
6
7   const handleLike = async () => {
8     // Update state immediately
9     setIsLiked(!isLiked);
10
11    try {
12      // Make the API call
13      await fetch(`/api/posts/${postId}/like`, { method: 'POST' });
14    } catch (err) {
15      // Revert state if an error occurs
16      setIsLiked(isLiked);
17      setError('Failed to update like status');
18    }
19  };
20
21  return (
22    <div>
23      <button onClick={handleLike}>
24        {isLiked ? 'Unlike' : 'Like'}
25      </button>
26      {error && <p>{error}</p>}
27    </div>
28  );
29 }
```

States used for setting optimistic updates

Optimistic state update

If an error occurs during the API call, it reverts isLiked to the original state.

The component immediately updates the isLiked state optimistically.

With **useOptimistic** hook: simplifies this pattern by automatically providing an optimistic state that can be **toggled** without additional error-handling code.



```
1 import { useOptimistic } from 'react';
2
3 function LikeButton({ postId }) {
4   const [isLiked, toggleLike] = useOptimistic(false);
5
6   const handleLike = async () => {
7     toggleLike(!isLiked); // Toggle optimistic state
8     await fetch(`/api/posts/${postId}/like`, { method: 'POST' });
9   };
10
11   return (
12     <button onClick={handleLike}>
13       {isLiked ? 'Unlike' : 'Like'}
14     </button>
15   );
16 }
```

- The **toggleLike** function immediately updates the UI, even while waiting for the server response.

useOptimistic makes optimistic updates less error-prone by allowing UI changes with minimal code, enhancing readability and **reducing boilerplate code for error handling**.

use ()

In earlier versions of React, handling **asynchronous** code, such as data fetching, within components typically involved **useEffect** and **useState**.

This approach often led to complex, nested logic, especially if we are to manage loading and error states.

The new **use()** hook allows you to directly await a promise within components, reducing the need for additional **useEffect** and **useState** code.

This makes it easier to fetch data in a synchronous style, leading to cleaner and more readable code.

Example: asynchronous data fetching using useEffect and useState hooks

```
1 import { useState, useEffect } from 'react';
2
3 function UserProfile() {
4   const [user, setUser] = useState(null);
5   const [loading, setLoading] = useState(true);
6
7   useEffect(() => {
8     const fetchUser = async () => {
9       try {
10         const response = await fetch('/api/user');
11         const data = await response.json();
12         setUser(data);
13       } finally {
14         setLoading(false);
15       }
16     };
17     fetchUser();
18   }, []);
19
20   if (loading) return <p>Loading ... </p>;
21
22   return (
23     <div>
24       <h1>{user.name}</h1>
25       <p>{user.email}</p>
26     </div>
27   );
28 }
```

The **useEffect** hook is used for running the fetch request on component mount.

loading state management is necessary to handle the UI during data fetching.

Example: using the new use() hook for asynchronous fetching

```
1 import { use } from 'react';
2
3 async function fetchUser() {
4   const response = await fetch('/api/user');
5   return response.json();
6 }
7
8 function UserProfile() {
9   const user = use(fetchUser()); // Directly await the data
10
11   return (
12     <div>
13       <h1>{user.name}</h1>
14       <p>{user.email}</p>
15     </div>
16   );
17 }
```

With use():

- Async calls are handled directly in the component body, reducing the need for useEffect and additional state variables.
- This approach streamlines data fetching, reduces boilerplate code, and is especially useful for server components or Suspense-based apps.

Use Cases for these new hooks:

- **useFormState()**: Ideal for applications that require dynamic forms with multiple fields, such as registration or profile forms. `useFormState()` reduces the setup required to manage input data and lets developers focus more on validation and form submission.
- **useFormStatus()**: Perfect for user-intensive applications where forms are frequently submitted, such as contact forms, checkout pages, or surveys. This simplifies tracking the form's state and handling any potential submission errors.
- **useOptimistic()**: Ideal for applications with real-time interactions, such as social media apps where users expect immediate feedback on actions like likes, votes, or toggling a setting.

- **useActionState()**: A use case for useActionState is managing the status of actions like adding an item to a shopping cart in an e-commerce app or submitting a form. This hook can track whether the action is pending, resolved, or rejected, allowing for real-time feedback, such as showing a loading state, confirming a successful addition, or displaying an error message if the action fails.
- **use()**: Particularly useful for components that need data from asynchronous sources, like user profiles or content from an API. It eliminates the need for useEffect and separate loading states, streamlining asynchronous operations.

Follow_for_more



 Somesh_Bhatnagar